Assignment 9: Advanced SQL Queries Using Joins and Subqueries

Hospital Management System - multi-table reports, summary statistics and subqueries

1. Objective

This assignment builds advanced reports that combine several tables using INNER and LEFT joins, correlated and non-correlated subqueries, IN / NOT EXISTS, and summary statistics. Each report states the business question, shows the SQL, the actual output and a short explanation.

Join aliases: a = APPOINTMENT/ADMISSIONS, p = PATIENTS, d = DOCTOR, dp = DEPARTMENTS, m = MEDICINES, pm = PRESCRIPTION_MEDICINE.

Part A: Reports Using Joins

Query 1: Upcoming appointment schedule (4-table join)

```
SELECT a.appointment_id, a.appointment_date, p.name AS patient, d.doctor_name AS doctor,
       dp.department_name AS department
FROM APPOINTMENT a
JOIN PATIENTS    p  ON a.patient_id    = p.patient_id
JOIN DOCTOR      d  ON a.doctor_id     = d.doctor_id
JOIN DEPARTMENTS dp ON d.department_id = dp.department_id
WHERE a.appointment_status = 'Scheduled'
ORDER BY a.appointment_date;
```

Output

appointment_id	appointment_date	patient	doctor	department
5017	2026-10-26	Mohan Das	Dr. Meera Nair	Cardiology
5018	2026-10-28	Karthik Naidu	Dr. Priya Menon	Pediatrics
5019	2026-10-30	Suresh Babu	Dr. Anil Kumar	Cardiology
5020	2026-11-02	Sneha Kulkarni	Dr. Lakshmi Iyer	General Medicine

4 row(s) returned

Explanation: Business question: who is booked with which doctor and department? INNER JOINs follow the foreign keys PATIENTS - APPOINTMENT - DOCTOR - DEPARTMENTS.

Query 2: Admission report with length of stay

```
SELECT a.admission_id, p.name AS patient, d.doctor_name AS doctor, a.room_number,
       a.admission_date, a.discharge_date,
       CAST(julianday(COALESCE(a.discharge_date, '2026-10-23')) - julianday(a.admission_date) AS INT)
           AS days_stayed
FROM ADMISSIONS a
JOIN PATIENTS p ON a.patient_id = p.patient_id
JOIN DOCTOR   d ON a.doctor_id  = d.doctor_id
ORDER BY days_stayed DESC;
```

Output

admission_id	patient	doctor	room_number	admission_date	discharge_date	days_stayed
7001	Mohan Das	Dr. Anil Kumar	A-201	2026-09-03	2026-09-09	6
7003	Fatima Begum	Dr. Suresh Rao	B-310	2026-09-14	2026-09-20	6
7005	Imran Pasha	Dr. Rajesh Gowda	A-102	2026-10-01	2026-10-07	6
7002	Suresh Babu	Dr. Meera Nair	A-205	2026-09-12	2026-09-16	4
7006	Pooja Sharma	Dr. Sunita Patil	C-210	2026-10-08	2026-10-12	4
7004	Karthik Naidu	Dr. Priya Menon	C-105	2026-09-18	2026-09-21	3
7007	Naveen Gowda	Dr. Mohammed Faiz	B-112	2026-10-16	2026-10-19	3
7008	Lakshmi Devi	Dr. Kavita Shetty	B-315	2026-10-20	NULL	3

8 row(s) returned

Explanation: For the patient still admitted (no discharge date) COALESCE uses today's date, 2026-10-23, so the stay is counted to date. In MySQL use DATEDIFF(COALESCE(a.discharge_date,'2026-10-23'), a.admission_date).

Query 3: Patients who have never been admitted (LEFT JOIN)

```
SELECT p.patient_id, p.name, p.Gender
FROM PATIENTS p
LEFT JOIN ADMISSIONS a ON p.patient_id = a.patient_id
WHERE a.admission_id IS NULL
ORDER BY p.patient_id;
```

Output

patient_id	name	Gender
1001	Ravi Shankar	Male
1002	Anitha Rao	Female
1004	Sneha Kulkarni	Female
1006	Divya Hegde	Female
1013	Aarav Menon	Male
1014	Rekha Joshi	Female
1015	Vikram Singh	Male

7 row(s) returned

Explanation: A LEFT JOIN keeps every patient; those with no matching admission get NULLs on the admission side, and the IS NULL filter selects exactly them (outpatient-only patients).

Query 4: Inpatient revenue by department

```
SELECT dp.department_name AS department, COUNT(b.bill_id) AS bills, SUM(b.total_amount) AS total_billed,
       AVG(b.total_amount) AS avg_bill
FROM BILL b
JOIN ADMISSIONS  a  ON b.admission_id  = a.admission_id
JOIN DOCTOR      d  ON a.doctor_id     = d.doctor_id
JOIN DEPARTMENTS dp ON d.department_id = dp.department_id
GROUP BY dp.department_name
ORDER BY total_billed DESC;
```

Output

department	bills	total_billed	avg_bill
Cardiology	2	147500.50	73750.25
Orthopedics	1	120000.00	120000.00
Gynecology	1	54300.00	54300.00
Neurology	1	48000.00	48000.00
General Medicine	1	18900.00	18900.00
Pediatrics	1	15750.00	15750.00

6 row(s) returned

Explanation: Follows BILL - ADMISSIONS - DOCTOR - DEPARTMENTS to attribute each inpatient bill to the responsible doctor's department. Outpatient bills (no admission) are excluded by the inner join.

Query 5: Outstanding payments with patient contact details

```
SELECT b.bill_id, p.name AS patient, p.ph_no, b.bill_date, b.total_amount, b.payment_status
FROM BILL b
JOIN PATIENTS p ON b.patient_id = p.patient_id
WHERE b.payment_status <> 'Paid'
ORDER BY b.total_amount DESC;
```

Output

bill_id	patient	ph_no	bill_date	total_amount	payment_status
9005	Imran Pasha	9845010005	2026-10-07	120000.00	Pending
9003	Fatima Begum	9845010008	2026-09-20	48000.00	Partial
9007	Naveen Gowda	9845010011	2026-10-19	18900.00	Pending

3 row(s) returned

Explanation: A collections report: all pending or partially paid bills with the phone number to follow up on.

Query 6: Prescription report with medicine count and estimated cost (many-to-many join)

```
SELECT rx.prescription_id, p.name AS patient, d.doctor_name AS doctor,
       COUNT(pm.medicine_id) AS medicines, SUM(pm.quantity * m.price) AS estimated_cost
FROM PRESCRIPTION rx
JOIN PATIENTS              p  ON rx.patient_id      = p.patient_id
JOIN DOCTOR                d  ON rx.doctor_id       = d.doctor_id
JOIN PRESCRIPTION_MEDICINE pm ON rx.prescription_id = pm.prescription_id
JOIN MEDICINES             m  ON pm.medicine_id     = m.medicine_id
GROUP BY rx.prescription_id, p.name, d.doctor_name
ORDER BY estimated_cost DESC;
```

Output

prescription_id	patient	doctor	medicines	estimated_cost
4006	Pooja Sharma	Dr. Sunita Patil	2	1480.00
4002	Mohan Das	Dr. Anil Kumar	3	699.00
4005	Suresh Babu	Dr. Meera Nair	3	549.00
4007	Naveen Gowda	Dr. Mohammed Faiz	2	295.00
4001	Ravi Shankar	Dr. Anil Kumar	2	189.00
4010	Anitha Rao	Dr. Lakshmi Iyer	1	180.00
4004	Karthik Naidu	Dr. Priya Menon	2	137.00
4009	Ravi Shankar	Dr. Anil Kumar	1	135.00
4008	Lakshmi Devi	Dr. Kavita Shetty	1	32.00
4003	Sneha Kulkarni	Dr. Lakshmi Iyer	1	25.00

10 row(s) returned

Explanation: PRESCRIPTION_MEDICINE resolves the M:N relationship; quantity multiplied by price gives each line cost, summed per prescription (price is today's catalogue price).

Part B: Reports Using Subqueries

Query 7: Bills above the average bill (non-correlated subquery)

```
SELECT b.bill_id, p.name AS patient, b.total_amount, b.payment_status
FROM BILL b
JOIN PATIENTS p ON b.patient_id = p.patient_id
WHERE b.total_amount > (SELECT AVG(total_amount) FROM BILL)
ORDER BY b.total_amount DESC;
```

Output

bill_id	patient	total_amount	payment_status
9005	Imran Pasha	120000.00	Pending
9001	Mohan Das	85000.00	Paid
9002	Suresh Babu	62500.50	Paid
9006	Pooja Sharma	54300.00	Paid
9003	Fatima Begum	48000.00	Partial

5 row(s) returned

Explanation: The inner query computes the average once (about 45,194); the outer query lists bills larger than that.

Query 8: Patients who were prescribed Aspirin (IN subquery)

```
SELECT patient_id, name
FROM PATIENTS
WHERE patient_id IN (SELECT rx.patient_id
                     FROM PRESCRIPTION rx
                     JOIN PRESCRIPTION_MEDICINE pm ON rx.prescription_id = pm.prescription_id
                     JOIN MEDICINES m ON pm.medicine_id = m.medicine_id
                     WHERE m.medicine_name LIKE 'Aspirin%');
```

Output

patient_id	name
1001	Ravi Shankar
1003	Mohan Das
1009	Suresh Babu

3 row(s) returned

Explanation: The subquery returns the ids of patients with an Aspirin line item; the outer query returns their details. Useful for drug-recall or interaction checks.

Query 9: Medicines never prescribed (NOT EXISTS)

```
SELECT m.medicine_id, m.medicine_name, m.stock_quantity
FROM MEDICINES m
WHERE NOT EXISTS (SELECT 1 FROM PRESCRIPTION_MEDICINE pm WHERE pm.medicine_id = m.medicine_id);
```

Output

medicine_id	medicine_name	stock_quantity
211	Omeprazole 20mg	2000

1 row(s) returned

Explanation: A correlated subquery tests, for each medicine, whether any prescription line uses it. Medicines with no usage are candidates for stock review.

Query 10: Patient(s) with the highest total billing (subquery on aggregates)

```
SELECT p.name AS patient, SUM(b.total_amount) AS total_billed
FROM BILL b
JOIN PATIENTS p ON b.patient_id = p.patient_id
GROUP BY p.patient_id, p.name
HAVING SUM(b.total_amount) = (SELECT MAX(patient_total)
                              FROM (SELECT SUM(total_amount) AS patient_total
                                    FROM BILL GROUP BY patient_id) t);
```

Output

patient	total_billed
Imran Pasha	120000.00

1 row(s) returned

Explanation: The innermost query totals bills per patient, the middle query finds the maximum of those totals, and HAVING keeps the patient(s) matching it (ties would all appear).

Part C: Summary Reports Across Many Tables

Query 11: Doctor workload summary (correlated scalar subqueries)

```
SELECT d.doctor_name AS doctor, dp.department_name AS department,
  (SELECT COUNT(*) FROM APPOINTMENT    x WHERE x.doctor_id = d.doctor_id) AS appointments,
  (SELECT COUNT(*) FROM ADMISSIONS     x WHERE x.doctor_id = d.doctor_id) AS admissions,
  (SELECT COUNT(*) FROM MEDICAL_RECORD x WHERE x.doctor_id = d.doctor_id) AS records,
  (SELECT COUNT(*) FROM PRESCRIPTION   x WHERE x.doctor_id = d.doctor_id) AS prescriptions
FROM DOCTOR d
JOIN DEPARTMENTS dp ON d.department_id = dp.department_id
ORDER BY appointments DESC, d.doctor_name;
```

Output

doctor	department	appointments	admissions	records	prescriptions
Dr. Anil Kumar	Cardiology	4	1	3	3
Dr. Lakshmi Iyer	General Medicine	4	0	2	2
Dr. Meera Nair	Cardiology	2	1	1	1
Dr. Priya Menon	Pediatrics	2	1	1	1
Dr. Rajesh Gowda	Orthopedics	2	1	1	0
Dr. Sunita Patil	Gynecology	2	1	1	1
Dr. Arjun Reddy	Pediatrics	1	0	0	0
Dr. Kavita Shetty	Neurology	1	1	1	1
Dr. Mohammed Faiz	General Medicine	1	1	1	1
Dr. Suresh Rao	Neurology	1	1	1	0

10 row(s) returned

Explanation: Each subquery counts one kind of related record for the current doctor. Counting in separate subqueries avoids the inflated totals that joining all four tables at once would produce.

Query 12: Patient 360 summary: visits, admissions, spending and balance

```
SELECT p.patient_id, p.name,
  (SELECT COUNT(*) FROM APPOINTMENT a WHERE a.patient_id = p.patient_id) AS appointments,
  (SELECT COUNT(*) FROM ADMISSIONS  a WHERE a.patient_id = p.patient_id) AS admissions,
  COALESCE((SELECT SUM(total_amount) FROM BILL b WHERE b.patient_id = p.patient_id), 0) AS total_billed,
  COALESCE((SELECT SUM(total_amount) FROM BILL b
            WHERE b.patient_id = p.patient_id AND b.payment_status <> 'Paid'), 0) AS due_amount
FROM PATIENTS p
ORDER BY total_billed DESC, p.patient_id;
```

Output

patient_id	name	appointments	admissions	total_billed	due_amount
1005	Imran Pasha	1	1	120000.00	120000.00
1003	Mohan Das	2	1	85000.00	0.00
1009	Suresh Babu	2	1	62500.50	0.00
1010	Pooja Sharma	1	1	54300.00	0.00
1008	Fatima Begum	1	1	48000.00	48000.00
1011	Naveen Gowda	1	1	18900.00	18900.00
1007	Karthik Naidu	2	1	15750.00	0.00
1004	Sneha Kulkarni	2	0	1500.00	0.00
1002	Anitha Rao	1	0	800.00	0.00
1001	Ravi Shankar	2	0	0.00	0.00
1006	Divya Hegde	1	0	0.00	0.00
1012	Lakshmi Devi	1	1	0.00	0.00
1013	Aarav Menon	1	0	0.00	0.00
1014	Rekha Joshi	1	0	0.00	0.00
1015	Vikram Singh	1	0	0.00	0.00

15 row(s) returned

Explanation: One row per patient combining appointment, admission and billing information. COALESCE turns NULL (no bills) into 0. due_amount is the full amount of pending and partially paid bills (the schema does not store the amount already paid on a partial bill).

Query 13: Low-stock medicines with total quantity prescribed

```
SELECT m.medicine_name, m.stock_quantity, COALESCE(SUM(pm.quantity), 0) AS units_prescribed
FROM MEDICINES m
LEFT JOIN PRESCRIPTION_MEDICINE pm ON m.medicine_id = pm.medicine_id
GROUP BY m.medicine_id, m.medicine_name, m.stock_quantity
HAVING m.stock_quantity < 500
ORDER BY m.stock_quantity;
```

Output

medicine_name	stock_quantity	units_prescribed
Insulin Glargine	80	2
Clopidogrel 75mg	150	30
Ceftriaxone 1g Inj	400	6

3 row(s) returned

Explanation: A LEFT JOIN keeps medicines with no prescriptions. HAVING limits the report to medicines with fewer than 500 units in stock - the items a pharmacist should reorder.

Conclusion

The reports show how the normalized design is recombined with joins: foreign keys provide the join paths, LEFT JOIN and NOT EXISTS find missing relationships, and subqueries compute comparisons against averages, maxima and per-row counts. The results give hospital management practical information on schedules, bed stays, revenue per department, unpaid bills, pharmacy usage and doctor workload.